

FINPILOT

Financial Workspace for Indian SMBs

Master Product, Company & Engineering Documentation

The source of truth for understanding FINPILOT

Version 1.0

September 2026

Internal Product and Engineering Documentation

Contents

1	Company and Product Overview	6
1.1	What is FINPILOT?	6
1.2	One-Sentence Definition	6
1.3	The Core Idea	6
1.4	Product Philosophy	7
2	The Problem FINPILOT Solves	8
2.1	The Current Problem	8
2.2	Fragmentation	8
2.3	Repetitive Work	9
2.4	Information vs. Understanding	9
3	Who FINPILOT Is For	11
3.1	Primary Users	11
3.1.1	Founders	11
3.1.2	Business Owners	11
3.1.3	Accountants	11
3.1.4	CFOs and Finance Heads	12
3.2	User Design Principle	12
4	Product Vision	13
4.1	Long-Term Vision	13
4.2	The Financial Intelligence Layer	13
5	Core Product Workflow	14
5.1	Step 1 — Connect	14
5.2	Step 2 — Process	14
5.3	Step 3 — Review	15
5.4	Step 4 — Understand	15
5.5	Step 5 — Act	15
6	Core Product Modules	16
6.1	1. Dashboard / Command Center	16
6.2	2. Transactions	16

6.3	3. AI Reconciliation	17
6.4	4. Invoices	17
6.5	5. Bills	18
6.6	6. Reports	18
6.7	7. AI Finance Assistant	18
6.8	8. Alerts and Exceptions	19
6.9	9. Approvals	19
6.10	10. Audit History	19
7	AI Philosophy	21
7.1	The Most Important AI Principle	21
7.2	What AI Can Do	21
7.3	What AI Should Not Do Automatically	21
7.4	AI Confidence	22
8	Financial Data Architecture	23
8.1	Conceptual Data Flow	23
8.2	Raw Data	23
8.3	Normalized Data	23
8.4	Deduplication	24
9	Product Data Model	25
9.1	Workspace	25
9.2	Transaction	25
9.3	Invoice	26
9.4	Bill	26
9.5	Reconciliation Record	27
9.6	Audit Record	27
10	Current Software Architecture	28
10.1	Current Technology Stack	28
10.2	Repository Structure	28
10.3	Frontend	29
10.4	Backend	29
10.5	Database	29
10.6	Important Existing Routes	29
11	Current Demo Environment	30
11.1	Purpose of the Demo	30
11.2	Demo Data	30
11.3	No Real Bank Connections	30
11.4	Why Deterministic Demo Data Matters	30

12 Landing Page and Public Product Experience	32
12.1 Landing Page Components	32
12.2 Landing Page Objective	33
12.3 Current Landing Page Limitation	33
13 Security and Trust Philosophy	34
13.1 Security Principles	34
13.2 Human Control	34
13.3 Auditability	35
14 Product Principles	36
14.1 Principle 1 — Financial Truth Comes First	36
14.2 Principle 2 — AI Assists, Humans Control	36
14.3 Principle 3 — Explainability Matters	36
14.4 Principle 4 — Do Not Hide Uncertainty	36
14.5 Principle 5 — One Workspace	36
14.6 Principle 6 — Product Before Complexity	36
14.7 Principle 7 — Financial UX Must Be Understandable	36
14.8 Principle 8 — Auditability by Design	37
15 Engineering Principles	38
15.1 Modularity	38
15.2 Separation of Responsibilities	38
15.3 Deterministic Financial Logic	39
15.4 AI Isolation	39
16 AI System Architecture	40
16.1 Context Construction	40
16.2 The AI Should Not Guess	40
16.3 Structured AI Outputs	41
17 Human-in-the-Loop Architecture	42
17.1 Example	42
17.2 Why This Matters	42
18 Application Architecture	43
18.1 Frontend Layer	43
18.2 API Layer	43
18.3 Service Layer	44
18.4 Database Layer	44
19 Development Environment	45
19.1 Current Development Model	45
19.2 Current Local Development	45

20 Current State vs. Target State	46
20.1 Current State	46
20.2 Target Product Direction	46
21 Roadmap Framework	48
21.1 Stage 1 — Product Foundation	48
21.2 Stage 2 — Financial Data	48
21.3 Stage 3 — Reconciliation	49
21.4 Stage 4 — Intelligence	49
21.5 Stage 5 — Control	49
21.6 Stage 6 — Integrations	50
22 What FINPILOT Is Not	51
22.1 Not Just an Accounting Dashboard	51
22.2 Not Just an AI Chatbot	51
22.3 Not an Autonomous Financial Robot	51
22.4 Not a Replacement for Financial Accountability	51
23 User Experience Philosophy	52
23.1 The Product Should Feel Intelligent	52
23.2 Progressive Disclosure	52
23.3 Financial Clarity	53
24 Design Language	54
24.1 Future Design Direction	54
25 Product Metrics	55
26 Developer Onboarding	56
26.1 Step 1 — Understand the Product	56
26.2 Step 2 — Understand the Core Workflow	56
26.3 Step 3 — Run the Application	56
26.4 Step 4 — Explore the Demo	56
26.5 Step 5 — Understand the Data	57
26.6 Step 6 — Understand AI Boundaries	57
27 Rules for Future Developers	58
27.1 Rule 1	58
27.2 Rule 2	58
27.3 Rule 3	58
27.4 Rule 4	58
27.5 Rule 5	58
27.6 Rule 6	58
27.7 Rule 7	58
27.8 Rule 8	58

27.9 Rule 9	59
27.10 Rule 10	59
28 Decision-Making Framework	60
29 Example End-to-End Scenario	61
29.1 Input	61
29.2 Processing	61
29.3 AI Processing	61
29.4 Human Review	62
29.5 Understanding	62
29.6 Outcome	62
30 Glossary	63
31 Open Product Decisions	64
31.1 Business	64
31.2 Integrations	64
31.3 AI	64
31.4 Security	65
31.5 Infrastructure	65
32 What Success Looks Like	66
33 The FINPILOT Mental Model	67
34 Final Principle	68
Document Control	69

1 Company and Product Overview

1.1 What is FINPILOT?

FINPILOT is an financial workspace designed for Indian small and medium-sized businesses .

The product is intended to bring different sources of business financial information into one intelligent environment.

These sources can include:

- bank transactions,
- accounting information,
- invoices,
- bills,
- spreadsheets,
- uploaded financial documents,
- payment information,
- and other business financial records.

Instead of requiring a business to continuously move between different systems, spreadsheets, accounting records, documents, and manual workflows, FINPILOT aims to create one place where financial information can be processed, reviewed, understood, and acted upon.

1.2 One-Sentence Definition

FINPILOT is an financial workspace that helps Indian SMBs bring fragmented financial data together, automate repetitive finance work, understand their financial position, and make better decisions while keeping humans in control.

1.3 The Core Idea

The fundamental idea behind FINPILOT is:

Financial data should not be fragmented across the business.

A business may have information distributed across:

- bank accounts,
- accounting software,
- Excel files,
- invoices,
- bills,
- payment platforms,
- documents,
- email,
- and manually maintained records.

FINPILOT is intended to reduce the operational friction caused by this fragmentation.

1.4 Product Philosophy

FINPILOT follows a simple conceptual model:

Connect → Process → Review → Understand → Act

The product should progressively transform raw financial information into useful business intelligence.

2 The Problem FINPILOT Solves

2.1 The Current Problem

Many SMB finance workflows are fragmented.

Financial information exists in multiple locations and often requires manual movement between systems.

A typical finance workflow can involve:

1. downloading bank statements,
2. importing or copying transactions,
3. maintaining accounting records,
4. matching payments,
5. checking invoices,
6. checking bills,
7. reconciling transactions,
8. categorizing expenses,
9. reviewing exceptions,
10. preparing reports,
11. and finally interpreting the numbers.

The individual tasks may appear simple.

The problem is the accumulation of these tasks.

2.2 Fragmentation

Financial information can be distributed across multiple sources:

Bank Accounts
Tally / Accounting Records
Excel
Invoices
Bills
Payment Platforms
Documents

FINPILOT's goal is to bring this information into a unified workspace.

2.3 Repetitive Work

Finance teams often spend significant operational effort on activities such as:

- finding matching transactions,
- checking duplicate records,
- categorizing transactions,
- comparing amounts,
- identifying exceptions,
- moving information between systems,
- answering repetitive financial questions,
- and preparing information for decision makers.

FINPILOT aims to use automation and AI to reduce this repetitive workload.

2.4 Information vs. Understanding

A business can possess a large amount of financial information without having a clear understanding of its financial position.

FINPILOT therefore separates two concepts:

1. **Financial data**
2. **Financial understanding**

The product should not simply store data.

It should help users answer questions such as:

- Where is money going?
- What are the major expenses?

- What transactions require attention?
- What has not been reconciled?
- What invoices or bills require attention?
- What is happening with cash flow?
- What financial patterns should the user understand?

3 Who FINPILOT Is For

FINPILOT is currently positioned around Indian SMBs and the people responsible for understanding and managing their finances.

3.1 Primary Users

3.1.1 Founders

Founders need visibility into the financial condition of the business without having to wait for a manually prepared report.

FINPILOT should help founders understand:

- where money is going,
- what the business is spending,
- cash position,
- important financial exceptions,
- and relevant financial trends.

3.1.2 Business Owners

Business owners need a simple way to understand the numbers without necessarily being finance specialists.

The interface should therefore translate financial complexity into understandable information.

3.1.3 Accountants

Accountants are expected to benefit from automation of repetitive finance operations.

Important areas include:

- reconciliation,
- transaction organization,
- duplicate detection,
- categorization,

- invoice and bill management,
- and exception review.

3.1.4 CFOs and Finance Heads

Finance leaders require visibility across financial operations.

FINPILOT should help provide a consolidated understanding of:

- cash,
- working capital,
- expenses,
- receivables,
- payables,
- reconciliation status,
- and financial performance.

3.2 User Design Principle

FINPILOT must not assume that every user has the same financial knowledge.

Therefore:

The product should expose financial complexity to finance professionals while presenting meaningful financial understanding to non-finance users.

4 Product Vision

4.1 Long-Term Vision

The long-term vision of FINPILOT is to become an intelligent financial operating environment for SMBs.

The product should evolve beyond simply displaying financial information.

It should increasingly help users:

- collect financial information,
- organize financial information,
- identify problems,
- understand financial conditions,
- answer financial questions,
- and take controlled financial actions.

4.2 The Financial Intelligence Layer

FINPILOT can be conceptually viewed as several layers.

1. **Data Layer**

Stores and organizes financial records.

2. **Processing Layer**

Validates, normalizes, deduplicates, and organizes financial information.

3. **Intelligence Layer**

Uses rules and AI to identify matches, categories, anomalies, exceptions, and insights.

4. **Human Review Layer**

Allows humans to verify or approve important outputs.

5. **Decision Layer**

Provides dashboards, reports, answers, alerts, and recommendations.

5 Core Product Workflow

The central FINPILOT workflow is:

**CONNECT → PROCESS → REVIEW → UNDERSTAND →
ACT**

5.1 Step 1 — Connect

Financial information enters FINPILOT.

Potential sources include:

- bank data,
- CSV files,
- accounting data,
- invoices,
- bills,
- financial documents,
- spreadsheets,
- and other supported sources.

The current public demo does not use real bank connections.

5.2 Step 2 — Process

Raw information must be transformed into structured records.

Processing can include:

- validation,
- normalization,
- duplicate detection,

- transaction organization,
- document processing,
- classification,
- and data consistency checks.

5.3 Step 3 — Review

AI and deterministic financial logic can identify items that require attention.

Examples include:

- possible transaction matches,
- duplicate transactions,
- unusual transactions,
- categorization suggestions,
- reconciliation exceptions,
- and document inconsistencies.

Humans review important outputs.

5.4 Step 4 — Understand

Processed financial information becomes useful business information.

FINPILOT can present:

- cash flow,
- income,
- expenses,
- receivables,
- payables,
- financial trends,
- exceptions,
- and reports.

5.5 Step 5 — Act

The final objective is not merely understanding.

Users should be able to take appropriate actions based on the information.

However, sensitive actions should remain subject to appropriate human control and approval.

6 Core Product Modules

6.1 1. Dashboard / Command Center

The dashboard is the central financial overview.

It should answer:

- What is happening financially?
- What needs attention?
- What has changed?
- What are the important numbers?

Possible dashboard information includes:

- total income,
- total expenses,
- net profit,
- cash balance,
- cash flow,
- outstanding items,
- reconciliation exceptions,
- and financial alerts.

6.2 2. Transactions

Transactions represent individual financial events.

The transaction system should eventually support:

- transaction records,
- categories,
- dates,

- amounts,
- counterparties,
- source information,
- matching,
- review status,
- and audit history.

6.3 3. AI Reconciliation

Reconciliation is one of the major FINPILOT capabilities.

The objective is to help identify whether financial records correspond to one another.
For example:

Bank Transaction

ABC Suppliers

INR 4,21,000

Potential Ledger Match

ABC Suppliers

INR 4,12,000

Difference

INR 9,000

AI Confidence

78%

Action

Review / Investigate

The system should never silently convert an uncertain AI match into an unquestioned financial truth.

6.4 4. Invoices

Invoices represent money owed to or from the business.

The invoice system should eventually provide visibility into:

- invoice records,
- counterparties,

- amounts,
- dates,
- payment status,
- outstanding amounts,
- and related transactions.

6.5 5. Bills

Bills represent business obligations.

FINPILOT should help users understand:

- what bills exist,
- what is due,
- what has been paid,
- what remains outstanding,
- and how bills relate to financial transactions.

6.6 6. Reports

Reports should transform financial records into structured financial understanding.

Examples include:

- income summaries,
- expense analysis,
- cash flow,
- receivables,
- payables,
- and other financial reports.

6.7 7. AI Finance Assistant

The AI Finance Assistant allows users to ask questions about their financial data.

Example:

User:

Show my top 5 expenses this month.

FINPILOT:

1. Raw Materials -- INR 2,45,000
2. Salaries -- INR 1,80,000
3. Rent -- INR 75,000
4. Marketing -- INR 42,000
5. Utilities -- INR 28,000

The important architectural principle is that the AI should answer based on the company's financial data rather than inventing financial facts.

6.8 8. Alerts and Exceptions

FINPILOT should surface information that requires attention.

Examples include:

- reconciliation mismatches,
- possible duplicate transactions,
- unusual transactions,
- overdue invoices,
- outstanding bills,
- and other defined financial exceptions.

6.9 9. Approvals

Humans must remain involved in sensitive financial operations.

Approval workflows provide a controlled mechanism for users to review and authorize important actions.

6.10 10. Audit History

Important financial operations should have traceability.

The audit system should eventually capture:

- what happened,
- when it happened,
- what entity was affected,

- what system or user initiated it,
- and what approval or review occurred.

7 AI Philosophy

AI is a major part of FINPILOT, but AI is not the financial source of truth.

7.1 The Most Important AI Principle

**The financial database and deterministic financial logic are the source of truth.
AI interprets, assists, detects, recommends, and explains.
Humans remain responsible for sensitive financial decisions and actions.**

This distinction is fundamental.

7.2 What AI Can Do

AI can assist with:

- transaction categorization,
- transaction matching,
- anomaly detection,
- exception identification,
- financial question answering,
- summarization,
- explanation,
- pattern identification,
- and recommendations.

7.3 What AI Should Not Do Automatically

AI should not silently:

- modify important financial records without authorization,

- approve sensitive financial actions,
- invent financial data,
- fabricate transaction history,
- present uncertain information as certain,
- or override deterministic financial records.

7.4 AI Confidence

Where appropriate, AI outputs should communicate uncertainty.

For example:

Potential Match Found

Confidence: 78%

Reason:

The counterparty matches, but the ledger amount differs from the bank transaction by INR 9,000.

Actions:

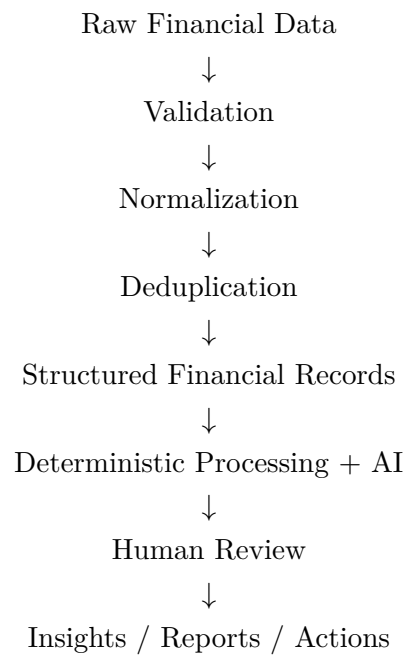
[Review] [Investigate]

This allows the AI to assist without pretending to be infallible.

8 Financial Data Architecture

8.1 Conceptual Data Flow

The conceptual data pipeline is:



8.2 Raw Data

Raw data is the information received from external sources.

The raw input should ideally be preserved where appropriate so that processing can be traced back to the original source.

8.3 Normalized Data

Different sources may represent the same concept differently.

Normalization creates a consistent internal representation.

For example:

"ABC SUPPLIERS LTD."

"ABC Suppliers"

"ABC SUPPLIER"

↓

Canonical Counterparty

"ABC Suppliers"

The exact normalization logic must be carefully designed before being used for production financial decisions.

8.4 Deduplication

The system should identify potentially duplicated financial records.

Duplicate detection should consider multiple properties rather than relying on a single field.

Potential attributes include:

- date,
- amount,
- counterparty,
- reference number,
- source,
- and transaction identifiers.

9 Product Data Model

The following is the conceptual model.

9.1 Workspace

A workspace represents a business environment.

A workspace may eventually contain:

- users,
- financial data,
- transactions,
- invoices,
- bills,
- reports,
- AI interactions,
- permissions,
- and audit history.

9.2 Transaction

A transaction represents a financial event.

Conceptually:

```
Transaction
-----
id
workspaceId
date
amount
currency
counterparty
```

category
description
source
status
createdAt
updatedAt

This is a conceptual model, not a finalized production schema.

9.3 Invoice

Conceptually:

Invoice

id
workspaceId
invoiceNumber
counterparty
amount
issueDate
dueDate
status
relatedTransactions
createdAt
updatedAt

9.4 Bill

Conceptually:

Bill

id
workspaceId
vendor
amount
issueDate
dueDate
status
relatedTransactions
createdAt
updatedAt

9.5 Reconciliation Record

Conceptually:

```
Reconciliation
-----
id
workspaceId
sourceRecord
candidateRecord
confidence
difference
reason
status
reviewedBy
reviewedAt
```

9.6 Audit Record

Conceptually:

```
AuditRecord
-----
id
workspaceId
actor
action
entityType
entityId
timestamp
metadata
```

Again, these schemas represent the product's conceptual direction.

The actual production schema must be finalized as implementation progresses.

10 Current Software Architecture

10.1 Current Technology Stack

The current FINPILOT application uses:

Layer	Technology
Frontend	React
Language	JavaScript
Build Tool	Vite
Styling	Tailwind CSS v3
Backend	Node.js
Server Framework	Express
Database	MongoDB
Database ODM	Mongoose

Table 10.1: Current FINPILOT technology stack

10.2 Repository Structure

The current application is organized broadly as:

```
project/
|
|-- client/
|   |
|   |-- src/
|       |
|       |-- components/
|           |   |
|           |   |-- landing/
|           |
|           |-- ...
|
|-- server/
|   |
```

```
|  | -- ...  
|  
|-- ...
```

10.3 Frontend

The frontend is implemented using React and JavaScript.

Vite is used as the frontend development/build environment.

Tailwind CSS v3 is used for styling.

10.4 Backend

The backend currently uses Node.js with Express.

The backend provides APIs consumed by the frontend.

10.5 Database

MongoDB is the current database.

Mongoose is used as the object data modeling layer.

10.6 Important Existing Routes

The current system includes:

- `/demo`
- `/api/waitlist`

The `/demo` route provides access to the existing interactive financial demo.

The `/api/waitlist` endpoint handles waitlist submissions.

11 Current Demo Environment

11.1 Purpose of the Demo

The public demo exists to communicate what FINPILOT can look like and how the product experience is intended to work.

It is not currently a production financial connection system.

11.2 Demo Data

The current demo uses deterministic simulated data.

The demo contains simulated examples including:

- transactions,
- invoices,
- bills,
- reconciliation exceptions,
- and simulated AI responses.

11.3 No Real Bank Connections

The current public demo does not connect to real bank accounts.

This distinction must remain obvious to users.

Important:

Demo financial data is simulated.

It must not be represented as real customer financial information.

11.4 Why Deterministic Demo Data Matters

The demo should produce predictable results.

This allows:

- reliable demonstrations,

- predictable UI behavior,
- reproducible bugs,
- easier development,
- and consistent product presentations.

12 Landing Page and Public Product Experience

The public-facing FINPILOT website is designed to introduce the product and move users toward either:

- the interactive demo,
- or early access / waitlist registration.

12.1 Landing Page Components

The current landing implementation has been modularized into components including:

- Navbar,
- Hero,
- Value Strip,
- Problem Section,
- Workflow Section,
- Product Showcase,
- Features Section,
- Demo CTA,
- Waitlist CTA,
- FAQ,
- Security section,
- Audience section,
- Waitlist Modal.

These components live under:

`client/src/components/landing/`

12.2 Landing Page Objective

The landing page should not attempt to explain every technical detail.

Its job is to quickly communicate:

1. what FINPILOT is,
2. who it is for,
3. what problem it solves,
4. how it works,
5. what the product looks like,
6. why the product is useful,
7. and how the user can experience it.

12.3 Current Landing Page Limitation

The current landing page is functional but has been identified as too basic from a product/brand perspective.

Future redesign work should therefore focus on making the product experience feel substantially more sophisticated while preserving clarity and the core FINPILOT positioning.

This means future landing-page work should not simply add more sections.

It should improve:

- visual storytelling,
- product visualization,
- perceived product depth,
- interaction,
- hierarchy,
- brand identity,
- and differentiation.

13 Security and Trust Philosophy

Financial software requires a fundamentally different approach to trust.

13.1 Security Principles

FINPILOT's current product philosophy includes:

- secure authentication,
- workspace-level access control,
- protected financial data,
- human approval for sensitive actions,
- audit history,
- and clear separation between demo data and real financial data.

13.2 Human Control

The system should not be designed around unrestricted automation.

Instead:

Automation + Verification + Human Control

This is particularly important when the system is dealing with:

- financial records,
- reconciliation,
- approvals,
- payments,
- or other financially sensitive actions.

13.3 Auditability

A future production system should make it possible to determine:

- what happened,
- why it happened,
- which data was involved,
- whether AI was involved,
- whether a human reviewed it,
- and what action followed.

FINPILOT should never optimize for automation at the cost of financial accountability.

14 Product Principles

These principles should guide future product and engineering decisions.

14.1 Principle 1 — Financial Truth Comes First

The underlying financial records and deterministic calculations are more authoritative than AI-generated interpretations.

14.2 Principle 2 — AI Assists, Humans Control

AI should reduce work without removing appropriate human oversight.

14.3 Principle 3 — Explainability Matters

When AI recommends an action or identifies an exception, users should understand why.

14.4 Principle 4 — Do Not Hide Uncertainty

A model confidence score or uncertain result should not be represented as a confirmed financial fact.

14.5 Principle 5 — One Workspace

FINPILOT should progressively reduce the need to move between disconnected financial tools.

14.6 Principle 6 — Product Before Complexity

New functionality should solve a real user problem.

Do not add complexity merely because a technology is available.

14.7 Principle 7 — Financial UX Must Be Understandable

The interface should communicate financial information clearly.

Complexity should exist in the system when necessary, not unnecessarily in the user's experience.

14.8 Principle 8 — Auditability by Design

Important financial operations should be traceable.

Auditability should not be added as an afterthought.

15 Engineering Principles

15.1 Modularity

The application should be built from understandable and reusable modules.

Avoid large components that combine:

- UI,
- business logic,
- API calls,
- state management,
- and financial calculations.

15.2 Separation of Responsibilities

Whenever possible:

```
UI
|
v
Frontend State / Service
|
v
API
|
v
Backend Service
|
v
Business Logic
|
v
Database
```

AI functionality should similarly remain separated from deterministic financial logic.

15.3 Deterministic Financial Logic

Financial calculations should be deterministic wherever possible.

For example:

`Net Profit = Total Income - Total Expenses`

Such calculations should not depend on an LLM.

An LLM may explain the result.

It should not be the calculator responsible for producing the authoritative number.

15.4 AI Isolation

AI integrations should be designed so that replacing the underlying model does not require rewriting the financial system.

Conceptually:

Financial Data

|

v

AI Context Builder

|

v

AI Service

|

v

Structured AI Result

|

v

Validation

|

v

Human / Application

16 AI System Architecture

The eventual AI architecture should be treated as a system rather than simply an API call.

16.1 Context Construction

When a user asks:

`"What were my largest expenses this month?"`

the system should first determine which financial records are relevant.

The conceptual flow is:

1. Understand the user's request.
2. Determine required financial data.
3. Query authoritative financial records.
4. Build structured context.
5. Send relevant context to the AI model.
6. Receive a response.
7. Validate the response where appropriate.
8. Present the result to the user.

16.2 The AI Should Not Guess

The AI assistant should not answer financial questions using general model knowledge when the answer depends on company-specific financial records.

For example:

BAD:

`"Your company spent approximately INR 10 lakh."`

GOOD:

"Based on the transactions recorded in your workspace,
your total expenses for the selected period are
INR X."

16.3 Structured AI Outputs

Where possible, AI systems should return structured results.

For example:

```
{  
  "type": "reconciliation_suggestion",  
  "confidence": 0.78,  
  "sourceTransactionId": "...",  
  "candidateTransactionId": "...",  
  "difference": 9000,  
  "reason": "Counterparty matches but amount differs."  
}
```

The exact production schema is TBD.

17 Human-in-the-Loop Architecture

FINPILOT should be designed around appropriate human review.

A useful conceptual model is:

AI Detection → AI Recommendation → Human Review → Approved State

For low-risk operations, automation may eventually be appropriate.

For high-risk operations, human approval should remain required.

17.1 Example

Suppose FINPILOT detects a potential reconciliation match.

The system may produce:

Potential match

Confidence: 92%

Bank:

ABC Suppliers

INR 4,21,000

Ledger:

ABC Suppliers

INR 4,21,000

The user may then approve the match.

The system records the approval.

17.2 Why This Matters

The product should not create a black-box financial automation system.

The objective is:

Make finance teams faster without making them blind.

18 Application Architecture

The application can be conceptually separated into the following layers.

18.1 Frontend Layer

Responsibilities:

- user interface,
- navigation,
- forms,
- dashboards,
- visualizations,
- user interaction,
- frontend state.

Technology:

- React,
- JavaScript,
- Vite,
- Tailwind CSS v3.

18.2 API Layer

Responsibilities:

- authentication requests,
- financial data requests,
- transaction APIs,
- invoice APIs,

- bill APIs,
- reconciliation APIs,
- AI APIs,
- and other application endpoints.

18.3 Service Layer

Business logic should live in services rather than being tightly coupled to HTTP route handlers.

Conceptually:

```
routes/  
  |  
  v  
controllers/  
  |  
  v  
services/  
  |  
  v  
repositories/models/  
  |  
  v  
MongoDB
```

The exact directory structure is subject to future implementation decisions.

18.4 Database Layer

MongoDB currently provides the persistence layer.

Mongoose provides schema and model management.

19 Development Environment

19.1 Current Development Model

The application currently consists of:

Frontend:

`client/`

Backend:

`server/`

The frontend runs through the Vite development environment.

The backend runs through the Node.js/Express server.

The landing page and demo can be developed independently while communicating with backend APIs where necessary.

19.2 Current Local Development

The landing page is currently accessible through the Vite development server.

The development setup has previously used:

`http://localhost:5173`

The backend has previously been used on:

`port 5000`

Exact development configuration should always be verified from the project's current environment configuration rather than assumed from this document.

20 Current State vs. Target State

One of the most important distinctions for every new team member is the difference between the current prototype/demo and the intended production product.

20.1 Current State

Currently established:

- React frontend.
- JavaScript codebase.
- Vite.
- Tailwind CSS v3.
- Node.js backend.
- Express.
- MongoDB.
- Mongoose.
- Interactive demo.
- Simulated financial data.
- Simulated AI behavior in the demo.
- Waitlist API.
- Public landing page.

20.2 Target Product Direction

The larger product is intended to support:

- real financial data ingestion,
- structured transaction management,

- reconciliation,
- invoices,
- bills,
- financial reporting,
- AI financial assistance,
- exception detection,
- approvals,
- audit trails,
- and controlled financial actions.

Important:

The target product capabilities described in this section should not be interpreted as proof that those capabilities are already implemented.

Every developer should check the actual repository and current product specification before assuming a feature exists.

21 Roadmap Framework

The exact roadmap is still an evolving product decision.

However, development can be organized into stages.

21.1 Stage 1 — Product Foundation

Focus:

- core application architecture,
- workspace model,
- authentication,
- user management,
- core database models,
- transaction infrastructure,
- basic dashboard.

21.2 Stage 2 — Financial Data

Focus:

- data ingestion,
- CSV imports,
- document processing,
- normalization,
- transaction management,
- invoices,
- bills.

21.3 Stage 3 — Reconciliation

Focus:

- matching engine,
- reconciliation workflows,
- exception management,
- AI-assisted matching,
- human review.

21.4 Stage 4 — Intelligence

Focus:

- AI Finance Assistant,
- financial question answering,
- anomaly detection,
- financial insights,
- alerts,
- natural-language explanations.

21.5 Stage 5 — Control

Focus:

- approvals,
- role-based permissions,
- audit trails,
- controlled financial actions,
- stronger security infrastructure.

21.6 Stage 6 — Integrations

Potential future integrations may include:

- banking,
- accounting systems,
- payment platforms,
- financial document systems,
- and other business tools.

The exact integrations and providers are not finalized in this document.

22 What FINPILOT Is Not

Understanding what the company is **not** building is equally important.

22.1 Not Just an Accounting Dashboard

A dashboard only displays information.

FINPILOT aims to help process, understand, and act on financial information.

22.2 Not Just an AI Chatbot

The AI Assistant is only one component of FINPILOT.

The underlying financial data system is more important than the conversational interface.

22.3 Not an Autonomous Financial Robot

FINPILOT should not blindly automate sensitive financial decisions.

Human oversight is a core product principle.

22.4 Not a Replacement for Financial Accountability

Automation should reduce repetitive work.

It should not remove accountability.

23 User Experience Philosophy

23.1 The Product Should Feel Intelligent

Intelligence should be visible through useful behavior, not merely through labels such as “AI-powered.”

For example, instead of:

AI Feature

the product should demonstrate intelligence through:

We found 17 possible reconciliation matches.

12 are high confidence.

3 require review.

2 appear to be duplicates.

23.2 Progressive Disclosure

Users should not see every technical detail at once.

The interface should reveal complexity when needed.

For example:

Transaction

```
|
+-- Category
|
+-- Match
|
+-- AI explanation
|
+-- Audit history
```

The basic interface remains simple.

Advanced information remains available.

23.3 Financial Clarity

Every important number should have context.

For example:

Net Profit

INR 4,13,000

vs. previous period

+12.4%

The exact analytics and comparison logic will depend on the final product specification.

24 Design Language

FINPILOT's current visual language uses:

- deep navy,
- blue primary actions,
- light blue backgrounds,
- clean white surfaces,
- strong typography,
- rounded cards,
- subtle borders,
- restrained shadows,
- and financial dashboard visualizations.

The design objective is a serious B2B financial product rather than a playful consumer application.

24.1 Future Design Direction

Future design work should make FINPILOT feel:

- sophisticated,
- intelligent,
- trustworthy,
- modern,
- premium,
- and product-driven.

However, visual complexity should not come at the expense of usability.

25 Product Metrics

The exact business KPIs have not yet been finalized.

However, product teams should eventually measure whether FINPILOT is reducing finance workload and increasing financial visibility.

Potential product measurements include:

- time saved during reconciliation,
- number of transactions processed,
- percentage of transactions automatically categorized,
- percentage of matches accepted,
- exception resolution time,
- AI answer accuracy,
- number of financial questions answered,
- user engagement with financial insights,
- and workspace retention.

These should be treated as potential metrics rather than finalized company KPIs.

26 Developer Onboarding

A new developer should follow this order.

26.1 Step 1 — Understand the Product

Read:

- this document,
- the current product specification,
- and the current repository README.

26.2 Step 2 — Understand the Core Workflow

Memorize:

CONNECT → PROCESS → REVIEW → UNDERSTAND → ACT

26.3 Step 3 — Run the Application

The developer should run:

- frontend,
- backend,
- database,
- demo,
- and waitlist flow.

26.4 Step 4 — Explore the Demo

The developer should understand:

- dashboard,

- transactions,
- reconciliation,
- invoices,
- bills,
- AI assistant,
- reports,
- and other currently implemented areas.

26.5 Step 5 — Understand the Data

Before changing financial functionality, developers must understand:

- where financial records originate,
- how they are stored,
- how they are transformed,
- how they are displayed,
- and how changes are recorded.

26.6 Step 6 — Understand AI Boundaries

Every developer working on AI features must understand:

- AI is not the financial source of truth,
- AI output can be uncertain,
- financial data must be grounded,
- important actions require appropriate validation,
- and sensitive actions require appropriate human control.

27 Rules for Future Developers

27.1 Rule 1

Do not introduce technology simply because it is popular.

Ask:

What problem does this solve for FINPILOT?

27.2 Rule 2

Do not put business-critical financial logic inside UI components.

27.3 Rule 3

Do not use an LLM for deterministic calculations.

27.4 Rule 4

Do not treat an AI suggestion as financial truth.

27.5 Rule 5

Do not remove auditability from important financial operations.

27.6 Rule 6

Do not silently change financial data.

27.7 Rule 7

Do not create fake production integrations merely to make a demo appear real.

27.8 Rule 8

Keep simulated/demo data clearly separated from real financial data.

27.9 Rule 9

Before adding a new feature, understand where it belongs in the overall FINPILOT architecture.

27.10 Rule 10

If a product decision has not been finalized, document it as a decision rather than silently making a permanent architectural assumption.

28 Decision-Making Framework

When deciding whether to build a feature, ask:

1. Does it solve a real finance problem?
2. Which user needs it?
3. What financial data does it require?
4. Does it affect financial truth?
5. Is deterministic logic sufficient?
6. Is AI actually useful?
7. Does a human need to review the result?
8. How will the action be audited?
9. What happens if the AI is wrong?
10. What happens if the data is incomplete?

A feature is not complete merely because its UI works.

For financial software, correctness and traceability are part of the feature.

29 Example End-to-End Scenario

Consider a fictional SMB using FINPILOT.

29.1 Input

The company imports financial information.

The system receives:

- bank transactions,
- ledger records,
- invoices,
- and bills.

29.2 Processing

FINPILOT validates and normalizes the data.

Duplicate records are identified.

Transactions are categorized.

29.3 AI Processing

The reconciliation engine identifies:

Bank transaction:

ABC Suppliers

INR 4,21,000

Potential ledger match:

ABC Suppliers

INR 4,12,000

The system identifies a difference:

$$4,21,000 - 4,12,000 = 9,000$$

The system flags the record for review.

29.4 Human Review

The finance user reviews the suggested match.

They can:

- accept,
- reject,
- or investigate.

29.5 Understanding

The dashboard updates the reconciliation status.

The user can then ask:

"What are my largest expenses this month?"

The AI assistant retrieves the relevant financial records and produces a grounded answer.

29.6 Outcome

The user has moved from:

Raw financial data

to:

Processed information

to:

Reviewed financial information

to:

Business understanding

This is the core FINPILOT product loop.

30 Glossary

Term	Meaning
FINPILOT	AI-powered financial workspace for Indian SMBs.
Workspace	A business environment containing users and financial information.
Transaction	An individual financial event.
Reconciliation	The process of comparing financial records and identifying matches or differences.
Exception	A financial record or condition that requires attention.
AI Finance Assistant	The conversational interface through which users can ask questions about financial data.
Command Center	The central dashboard for financial visibility and important actions.
Human-in-the-loop	A system design in which humans review or approve appropriate AI/system outputs.
Audit Trail	A record of important actions and changes.
Simulated Data	Data created for demonstrations and testing rather than representing real customer financial information.
Deterministic Logic	Logic that produces predictable results from defined inputs.
AI Confidence	A representation of how strongly an AI system supports a particular recommendation or classification.

31 Open Product Decisions

The following areas should be treated as open decisions until formally finalized.

31.1 Business

- Final pricing model.
- Subscription structure.
- Free vs paid functionality.
- Enterprise pricing.
- Exact target SMB segment.
- Sales model.
- Partnership strategy.

31.2 Integrations

- Exact banking providers.
- Exact accounting integrations.
- Payment platform integrations.
- Document providers.
- Integration rollout sequence.

31.3 AI

- Exact model providers.
- Model routing strategy.
- Retrieval architecture.
- Evaluation framework.

- AI cost controls.
- Production confidence thresholds.

31.4 Security

- Final authentication architecture.
- Production authorization model.
- Encryption architecture.
- Secrets management.
- Compliance requirements.
- Data retention policies.

31.5 Infrastructure

- Production hosting.
- Deployment architecture.
- CI/CD.
- Observability.
- Backup strategy.
- Disaster recovery.

Open decisions should be documented explicitly.

A developer should never assume that an unresolved decision is already company policy.

32 What Success Looks Like

The ultimate goal of FINPILOT is not to create more financial software.

The goal is to make financial operations easier to understand and operate.

A successful FINPILOT experience should allow a business owner or finance professional to move from:

“I have financial data everywhere.”

to:

“My financial information is organized.”

then:

“I know what needs attention.”

then:

“I understand what is happening.”

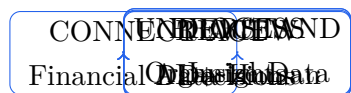
and ultimately:

“I can make better financial decisions.”

This is the fundamental product promise.

33 The FINPILOT Mental Model

Every team member should be able to understand FINPILOT using the following mental model:



Underneath all of this is the financial data layer.

Above it is the user experience.

AI operates across the system as an intelligence layer.

Humans remain in control of sensitive decisions.

34 Final Principle

FINPILOT should always be thought of as:

A Financial Intelligence Workspace

not merely a dashboard, not merely accounting software, and not merely an AI chatbot.

The system combines:

Financial Data + Automation + AI + Human Review + Financial Understanding

The product exists to turn fragmented financial information into a system that businesses can understand and operate.

FINPILOT

Connect the data.

Process the work.

Review intelligently.

Understand the business.

Act with confidence.

Document Control

Document	FINPILOT Master Product Documentation
Version	1.0
Date	September 2026
Purpose	Company, Product and Engineering Onboarding
Audience	Founders, Developers, Designers, AI Engineers, Product Teams
Status	Living Document

Maintenance Rule

This document should evolve with the product.

When a major product or architectural decision is finalized, the relevant section should be updated.

The repository, implementation, and this document should remain aligned.

If the implementation and this document disagree, the discrepancy should be explicitly resolved rather than silently ignored.